Member 3 Report RESTful API Documentation and Advanced Features

3.0 RESTful API Documentation

Member 3 implemented the Events API and the event management frontend. The Events API is a resource handled by the shared PHP router at api/index.php, uses the shared MySQL database, and returns the project's standard JSON response format. The frontend calls the API to browse, create, view, update, and delete events.

3.1 API Base URL

For the shared XAMPP setup, the API base URL is:

```
http://localhost/event-ticketing/api
```

The frontend defaults to this URL. If the project is installed under a different folder, the API URL can be changed using the frontend's API configuration control. When the frontend is served with PHP's built-in server, Apache and MySQL must still be running to serve the API and database.

3.2 Endpoints and HTTP Methods

Method	Endpoint	Authentication and access	Description
GET	/events	Public for published events. Admin and organiser tokens can view every status.	List events; supports pagination and filters.
GET	/events/{id}	Public for published events. Admin and organiser tokens can view every status.	Retrieve an event by ID.
POST	/events	Admin or organiser	Create an event.
PUT	/events/{id}	Admin or the event's owning organiser	Update an event.
PATCH	/events/{id}	Admin or the event's owning organiser	Partially update an event.
DELETE	/events/{id}	Admin or the event's owning organiser	Delete an event if no booking references prevent deletion.

The list endpoint supports these query parameters:

Parameter	Purpose
page	Page number; defaults to 1.
limit	Results per page; defaults to 10 and is capped at 100.
search	Search event title and description.
category	Filter by category.
venue_id	Filter by venue ID.
status	Filter by draft, published, cancelled, or completed.
organiser_id	Filter by organiser ID.

Example request:

```
GET /event-ticketing/api/events?page=1&limit=10&search=music&status=published
```

3.3 Authentication Requirements

Event reads are public for published events. If a guest or customer requests an unpublished event by ID, the API returns 404. Admins and organisers can view unpublished events for management.

Creating, updating, and deleting events requires a valid JWT. The client sends it in this header:

```
Authorization: Bearer <JWT>
```

The JWT is obtained from the shared POST /auth/login endpoint. The Events API uses the shared require_auth() and require_role() middleware. Admins can manage any event; organisers can manage only events they own. Only admins can assign or reassign an event to a different organiser.

3.4 Request JSON

The following example creates a published event. title, venue_id, event_date, start_time, end_time, and total_tickets are required. description, category, ticket_price, available_tickets, and status are optional. Category defaults to Community, ticket price to 0, available tickets to the total, and status to draft.

```
POST /event-ticketing/api/events
Content-Type: application/json
Authorization: Bearer <JWT>

{
  "title": "Johor Music Night",
  "description": "An evening of live music.",
  "category": "Music",
  "venue_id": 1,
  "event_date": "2026-11-20",
  "start_time": "19:00",
  "end_time": "22:00",
  "ticket_price": 35.00,
  "total_tickets": 300,
  "available_tickets": 300,
  "status": "published"
}
```

An administrator may also include organiser_id to assign the event to an existing admin or organiser. Other roles cannot assign events to another user.

For PUT or PATCH, send one or more supported event fields in the JSON body. The event ID is part of the URL. DELETE needs no request body.

3.5 Response JSON

Successful responses use the shared envelope. A successful create returns HTTP 201 and the event data:


```
{
  "success": true,
  "message": "Event created.",
  "data": {
    "id": 12,
    "title": "Johor Music Night",
    "category": "Music",
    "venue_id": 1,
    "venue_name": "City Hall",
    "venue_city": "Johor Bahru",
    "organiser_id": 2,
    "organiser_name": "Hafiz Ahmad",
    "event_date": "2026-11-20",
    "start_time": "19:00:00",
    "end_time": "22:00:00",
    "ticket_price": "35.00",
    "total_tickets": 300,
    "available_tickets": 300,
    "status": "published"
  }
}
```


The list response places event records in data and includes pagination metadata:


```
{
  "success": true,
  "message": "Events retrieved.",
  "data": [],
  "pagination": {
    "page": 1,
    "limit": 10,
    "total": 0,
    "total_pages": 0
  }
}
```


Validation errors use HTTP 422 and identify invalid fields:


```
{
  "success": false,
  "message": "Validation failed.",
  "errors": {
    "end_time": "End time must be later than start time."
  }
}
```

3.6 HTTP Status Codes

Code	Meaning in the Events API
200	Event list or detail returned, event updated, or event deleted.
201	Event created.
400	Invalid event ID or empty update request.
401	Missing, invalid, or expired JWT on a protected request.
403	Role or event ownership does not permit the requested action.
404	Event or venue not found, or an unpublished event is hidden from a guest/customer.
405	HTTP method is not supported for the endpoint.
409	Venue unavailable, event schedule overlaps, or bookings prevent deletion.
422	Request fields fail validation.
500	Unexpected server error, returned through the shared exception handler.

8.0 Additional and Advanced Features

8.1 Event Search, Filtering, and Pagination

The list endpoint can search titles and descriptions and filter by category, venue, status, and organiser. It returns the current page, page size, total result count, and number of pages. The page size is capped at 100 to keep responses bounded.

8.2 Role-Based Access and Ownership

Guests and customers can browse published events. Organisers and admins can also view drafts, cancelled events, and completed events. All write operations require a JWT. An organiser can update or delete only events assigned to their own account, while an admin can manage events across organisers. Organiser reassignment is restricted to admins and validated against the target user's role.

8.3 Venue Availability and Schedule Conflict Checks

Before an event is created, the API checks that its venue exists and has available status. It then checks for a draft or published event at the same venue, on the same date, with overlapping times. A conflict returns HTTP 409 and includes the conflicting event details. The same checks run when an update changes an event's venue, date, time, or status into a schedulable state.

8.4 Input Validation and Booking-Safe Deletion

The API validates date and time formats, event status, text lengths, positive ticket totals, non-

negative available tickets, and ticket price precision. It also ensures available tickets do not exceed total tickets and the end time follows the start time. Deletion is blocked if confirmed bookings exist; the database foreign key protects any event with remaining linked bookings. These checks return consistent JSON errors rather than raw database messages.

8.5 Event Management Frontend

The Member 3 frontend in frontend/member3-event-management/ provides public event browsing and role-aware event management. Visitors can search, filter, paginate, and open event details. Admins and organisers can use forms to create and edit events, and can delete eligible events. Venue choices are loaded from the shared Venues API. The page stores the selected API URL and login session in browser storage and can display API validation and conflict messages.

8.6 API Verification Evidence

The PHP route passed php -l, and the frontend JavaScript passed node --check. These are syntax checks only. Runtime requests against the shared XAMPP/MySQL database have not yet been completed in this workspace. Capture Postman or browser evidence for the following before submitting the final combined report:

Scenario	Expected result	Evidence status
GET published event list as guest	200 with published events and pagination	Capture during local API run
POST valid event as organiser	201 with created event	Capture during local API run
POST invalid event data	422 with field errors	Capture during local API run
POST event into unavailable or overlapping venue schedule	409 conflict	Capture during local API run
Update another organiser's event	403 for an organiser	Capture during local API run
Delete event with linked booking	409 conflict	Capture during local API run

8.7 Member 3 Contribution Files

- api/routes/events.php – Events API endpoints, validation, permissions, search, pagination, and venue/booking conflict handling.
- frontend/member3-event-management/index.html – event browsing and management interface.
- frontend/member3-event-management/style.css – frontend styling.
- frontend/member3-event-management/app.js – API client, authentication session handling, browse views, and CRUD interactions.
- frontend/member3-event-management/README.md – frontend setup and endpoint summary.